

Pattern Printing

DEFINITION

Pattern printing is the practice of using nested loops to arrange characters into a shape — a square, a triangle, a pyramid, a diamond. The shape itself isn't the point: this topic exists to train you to see how the outer loop (row), the inner loop (column), and the relationship between them (*i* and *j*) control every character that gets printed. Every hard DSA loop problem later builds on this exact instinct.

The Golden Rule

Every pattern has two loops: the outer loop decides which row you're on, and the inner loop decides what gets printed in that row. Before writing any code, answer three questions on paper first:

Question	Decides
How many rows?	Range of the outer loop (<i>i</i>)
How many things per row?	Range of the inner loop (<i>j</i>) — often depends on <i>i</i>
What gets printed?	<code>*</code> , a number, a letter, or a space — depends on <i>i</i> and/or <i>j</i>

★ NOTE

- Always write the outer loop first and ask 'how many rows are there?' — that number becomes your outer loop's limit.
- Then, for ONE row, work out the inner loop by hand before generalizing it with *i*.
- If the inner loop's limit changes with *i* (like 1, 2, 3...), that's your biggest clue it's a triangle-family pattern, not a square.

1. Square / Rectangle Pattern

The simplest pattern: every row prints the same number of columns, so the inner loop's limit never depends on *i*.

PYTHON

```
n = 4
for i in range(n):
    for j in range(n):
        print("*", end=" ")
    print()
```

Output

```
* * * *
* * * *
* * * *
* * * *
```



```
* * * *
* * * *
* * * *
* * * *
```

★ NOTE

- The inner loop's range never looks at i — it's always $\text{range}(n)$, fixed.
- This is the pattern every other one deviates from: as soon as the inner range starts depending on i , you get a triangle instead of a rectangle.

2. Right-Angled Triangle (Half Pyramid)

Now the inner loop's limit depends on the current row i — row 1 prints 1 star, row 2 prints 2 stars, and so on.

PYTHON

```
n = 5
for i in range(1, n + 1):
    for j in range(1, i + 1):
        print("*", end=" ")
    print()
```

Output

```
*
* *
* * *
* * * *
* * * * *
```



```
*
* *
* * *
* * * *
* * * * *
```

Swap $\text{range}(1, i + 1)$ for $\text{range}(1, n - i + 2)$ and you get the mirror image: a triangle that shrinks as it goes down.

3. Number Triangle

Same shape as above, but instead of printing a fixed '*', we print the value of the loop variable itself.

PYTHON

```
n = 5
for i in range(1, n + 1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```

Output

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

★ NOTE

- Print i instead of j inside the loop to get repeated row numbers: 1 / 2 2 / 3 3 3 / 4 4 4 4 — same loops, only the printed value changes.

4. Inverted Right Triangle

Flip the row count so it decreases: count the outer loop down from n instead of up to it.

PYTHON

```
n = 5
for i in range(n, 0, -1):
    for j in range(1, i + 1):
        print("*", end=" ")
    print()
```

Output

```
* * * * *
* * * *
* * *
* *
*
```

```

*   *   *   *   *
*   *   *   *
*   *   *
*   *
*

```

5. Pyramid (Centered Triangle)

A true pyramid needs leading spaces to center each row, on top of the stars. Each row i needs $(n - i)$ spaces followed by $(2 * i - 1)$ stars — work this out on paper for $n = 4$ before trusting the formula.

PYTHON

```

n = 5
for i in range(1, n + 1):
    for j in range(1, n - i + 1):
        print(" ", end="")
    for j in range(1, 2 * i):
        print("*", end="")
    print()

```

Output

```

  *
 ***
*****
*****
*****

```

```

      *
     * * *
    * * * * *
   * * * * * *
  * * * * * * *
 * * * * * * *

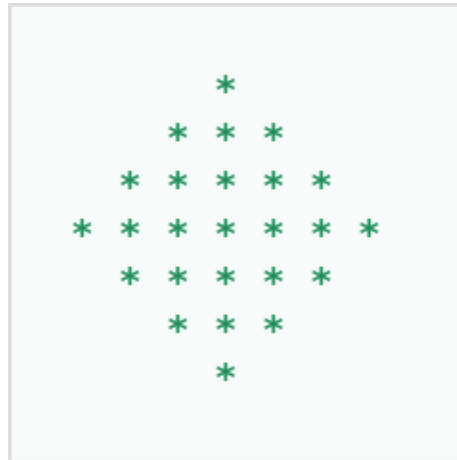
```

★ NOTE

- It's normal for a single row to need two separate inner loops back to back — one for spaces, one for the visible characters.
- Space count shrinks as i grows $(n - i)$; star count grows as i grows $(2 * i - 1)$ — that's what makes the sides slant inward at the top.
- Pythonic shortcut: the whole row can also be written in one line as `print(" " * (n - i) + "*" * (2 * i - 1))` — worth knowing, but trace the loop version first.

6. Inverted Pyramid


```
*****
*****
***
*
```

**★ NOTE**

- Notice the bottom half is the exact same pyramid loop, just with *i* counting down from *n*-1 instead of up to *n*.
- Most 'complex-looking' patterns (diamonds, hourglasses, butterflies) are two familiar patterns glued together — spot the two halves before coding.

8. Floyd's Triangle (Continuous Numbers)

Instead of restarting the count every row, a single counter keeps increasing across the entire pattern.

PYTHON

```
n = 4
num = 1
for i in range(1, n + 1):
    for j in range(1, i + 1):
        print(num, end=" ")
        num += 1
    print()
```

Output

```
1
2 3
4 5 6
7 8 9 10
```

```

1
2  3
4  5  6
7  8  9  1 0

```

★ NOTE

- The counter (num) lives outside both loops and is created once — it must NOT reset inside the outer loop, or every row would restart from 1.

9. Character Pattern

Characters work the same as numbers — use `chr()` and `ord()` to convert a numeric offset into a letter. This trick shows up constantly.

PYTHON

```

n = 5
for i in range(1, n + 1):
    for j in range(1, i + 1):
        ch = chr(ord('A') + j - 1)
        print(ch, end=" ")
    print()

```

Output

```

A
A B
A B C
A B C D
A B C D E

```

```

A
A  B
A  B  C
A  B  C  D
A  B  C  D  E

```

10. Hollow Square

A classic interview favorite. The border cells (first row, last row, first column, last column) get a '*'; everything else gets a space. Notice this needs an `if/else` INSIDE the inner loop instead of a separate loop for spaces.

PYTHON

```

n = 5

```

```

for i in range(1, n + 1):
    for j in range(1, n + 1):
        if i == 1 or i == n or j == 1 or j == n:
            print("*", end=" ")
        else:
            print(" ", end=" ")
    print()

```

Output

```

* * * * *
*         *
*         *
*         *
*         *
* * * * *

```

```

*   *   *   *   *
*               *
*               *
*               *
*               *
*   *   *   *   *

```

★ NOTE

- This is the first pattern where the DECISION to print '*' or ' ' happens inside a single inner loop via a condition, rather than two separate loops.
- The border condition ($i == 1$ or $i == n$ or $j == 1$ or $j == n$) is worth memorizing — it appears in hollow rectangles, borders, and matrix-boundary problems constantly.

11. Hollow Right Triangle

Same hollow idea, but applied to a triangle instead of a square: only the first row, the last row, and the two slanted/vertical edges get a star.

PYTHON

```

n = 5
for i in range(1, n + 1):
    for j in range(1, i + 1):
        if j == 1 or j == i or i == n:
            print("*", end=" ")
        else:
            print(" ", end=" ")
    print()

```

Output

```

*
* *
*  *
*   *
* * * * *

```




12. Butterfly Pattern

Two triangles facing each other, mirrored left-right, stitched top half and bottom half — same building blocks as the diamond, just wider because there's no gap-closing to the centre point.

PYTHON

```

n = 5
# top half
for i in range(1, n + 1):
    for j in range(1, i + 1):
        print("*", end="")
    for j in range(1, 2 * (n - i) + 1):
        print(" ", end="")
    for j in range(1, i + 1):
        print("*", end="")
    print()
# bottom half (mirror of the top, i counts back down)
for i in range(n, 0, -1):
    for j in range(1, i + 1):
        print("*", end="")
    for j in range(1, 2 * (n - i) + 1):
        print(" ", end="")
    for j in range(1, i + 1):
        print("*", end="")
    print()

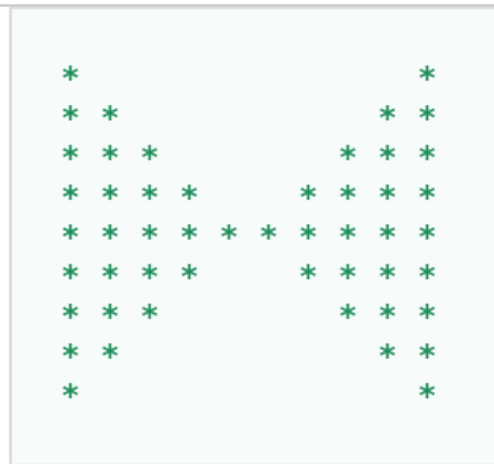
```

Output

```

*           *
**          **
***         ***
****        ****
*****       *****
*****       *****
****        ****
***         ***
**          **
*           *

```



★ NOTE

- Left wing, middle gap, right wing — three separate inner loops in a row, same row.
- The bottom half is just the top half's outer loop run backwards, exactly like the diamond's second half.

13. Pascal's Triangle

Each entry is nCr — the number of ways to choose r items from n . Rather than compute factorials, build each row from the previous one: multiply by $(\text{row} - \text{col})$ and divide by col as you go, which avoids overflow for longer rows.

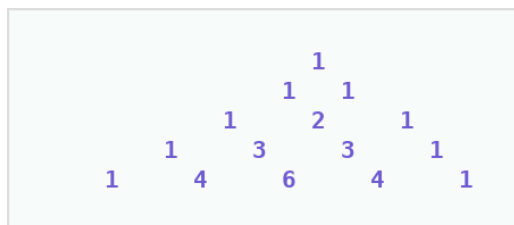
PYTHON

```
n = 5
for i in range(n):
    print(" " * (n - i - 1), end="")
    val = 1
    for j in range(i + 1):
        print(val, end=" ")
        val = val * (i - j) // (j + 1)
    print()
```

Output

```

    1
   1 1
  1 2 1
 1 3 3 1
1 4 6 4 1
```



★ NOTE

- The value printed isn't a function of i and j alone — it's built iteratively as you move across the row, which is the bridge from simple loop patterns into DP-style row-by-row construction.

14. Zigzag Pattern

A 3-row wave built from a single running condition. Track which 'peak' each column belongs to and alternate the row a star lands on.

PYTHON

```
n = 12
for i in range(3):
    for j in range(n):
        if i == 0 and j % 4 == 0:
            print("*", end="")
        elif i == 1 and (j % 4 == 1 or j % 4 == 3):
            print("*", end="")
        elif i == 2 and j % 4 == 2:
            print("*", end="")
        else:
            print(" ", end="")
    print()
```

Output

```
*   *   *   *
 * * * * *
  *   *   *
```



★ NOTE

- It's the first pattern where the printed character depends on (i, j) through a modulo condition rather than a clean range — good practice for reading tricky conditions before you hit them in a real problem.

★ NOTE

- Mixing up `range(1, n)` and `range(1, n + 1)` — Python's `range()` excludes its stop value, so this is the #1 source of an off-by-one row or column.
- Forgetting `end=" "` (or `end=""`): `print()` adds a newline by default, so without it every value lands on its own line instead of the same row.
- Forgetting the plain `print()` after the inner loop finishes, which merges every row onto a single line instead of starting a new one.
- For pyramids: printing the space-loop and star-loop in the wrong order, or forgetting one of them entirely.

How to Approach Any New Pattern

Step	What to do
1	Draw the shape for a small n (like n = 4) on paper.
2	Count rows -> that's your outer loop's range.
3	For each row, count spaces and symbols separately, and write them as a function of i.
4	Translate each function directly into an inner loop's range().
5	Trace the code by hand for n = 4 before running it.

Tomorrow's practice set: number pyramids with repeated row values, the diamond-of-numbers pattern, and a spiral matrix print. Same method — rows first, then spaces, then symbols.

ROUGH WORK

ROUGH WORK

ROUGH WORK

End of Notes — Pattern Printing

NextGen Data Minds